Grouped-Query Attention & KV-cache benchmark

Part 3 · Data & Payload Specialist — Tahjib Al Sakin · file `03_GQA_and_KVCache_Benchmark.py` · run: `python 03_GQA_and_KVCache_Benchmark.py`

`GQAAttention` with configurable H_Q : H_KV (MHA, GQA, MQA as special cases), grouping query heads so K and V are never copied; a stateful `KVCache` that appends one token per decoding step. Exactness checks against a full causal pass, repeat_interleave and Part 1. Benchmark over N ∈ {1024, 4096, 16384, 32768}: KV-cache size, bandwidth at 50 tok/s, H100 memory-bound lower bound and measured CPU decode latency.

Run output

```
$ python 03_GQA_and_KVCache_Benchmark.py
================================================================================
USSE NOOSPHERE · SUBSYSTEM 03 · PART 3: GQA & KV-CACHE

=== Correctness ===
cached token-by-token decoding vs full causal pass: max |diff| = 4.77e-07
GQA grouped view vs repeat_interleave:              max |diff| = 0.00e+00
GQA with H_kv = H_q vs Part 1 MultiHeadSelfAttention: max |diff| = 1.79e-07
[PASS] KV-cache and GQA are exact.

=== Why cache? Generating 256 tokens with one layer (CPU) ===
recompute prefix every step: 633 ms   ·   with KV-cache: 71 ms   ·   8.9× faster

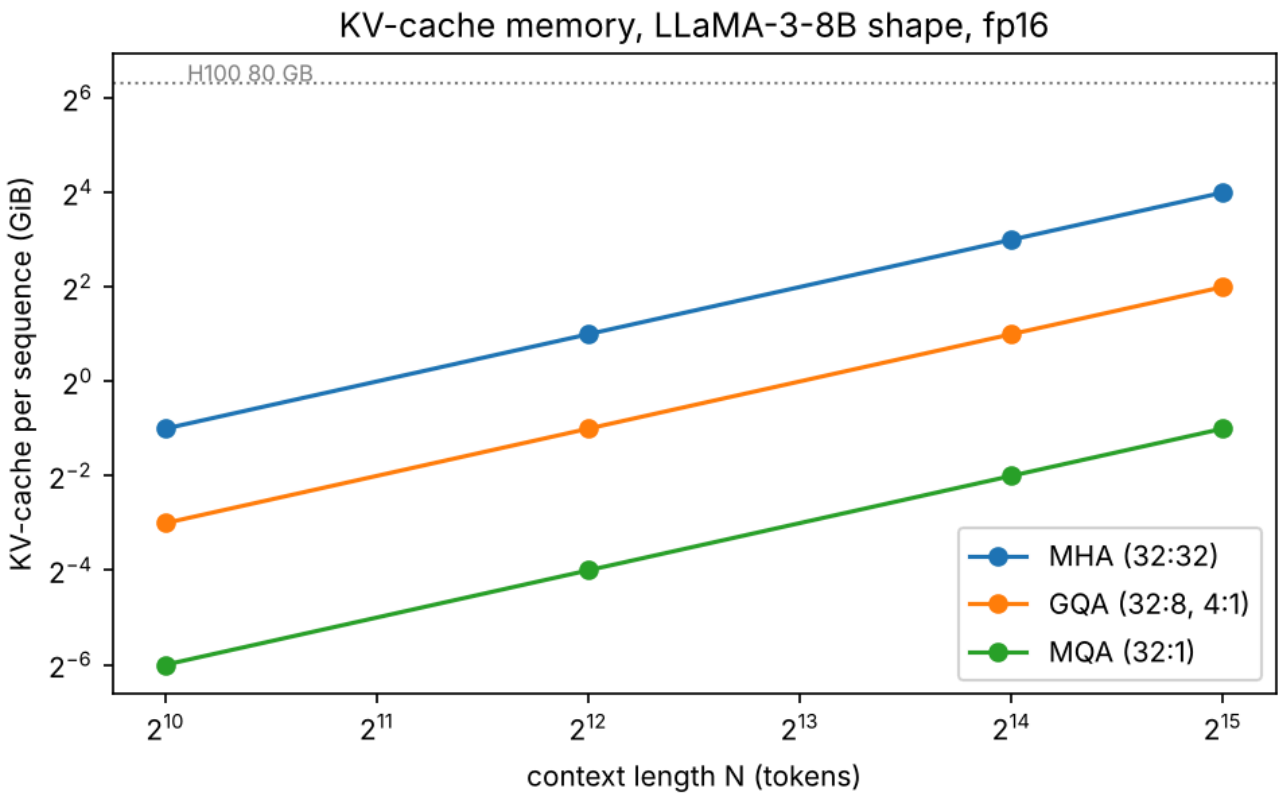
=== Benchmark: MHA vs GQA vs MQA ===
KV-cache for a LLaMA-3-8B-shaped model (32 layers, 32 query heads, d_head 128, fp16, batch 1)
Variant          |      N |   KV-cache | BW @50 tok/s | H100 KV read | CPU attn step
                 |        |       (MB) |       (GB/s) |   (ms/token) |    (ms/layer)
--------------------------------------------------------------------------------
MHA (32:32)      |   1024 |        512 |         26.8 |        0.160 |         12.88
MHA (32:32)      |   4096 |      2,048 |        107.4 |        0.641 |         17.88
MHA (32:32)      |  16384 |      8,192 |        429.5 |        2.564 |         43.92
MHA (32:32)      |  32768 |     16,384 |        859.0 |        5.128 |         75.38
GQA (32:8, 4:1)  |   1024 |        128 |          6.7 |        0.040 |          9.24
GQA (32:8, 4:1)  |   4096 |        512 |         26.8 |        0.160 |         10.30
GQA (32:8, 4:1)  |  16384 |      2,048 |        107.4 |        0.641 |         24.82
GQA (32:8, 4:1)  |  32768 |      4,096 |        214.7 |        1.282 |         30.63
MQA (32:1)       |   1024 |         16 |          0.8 |        0.005 |          6.16
MQA (32:1)       |   4096 |         64 |          3.4 |        0.020 |          8.08
MQA (32:1)       |  16384 |        256 |         13.4 |        0.080 |          8.89
MQA (32:1)       |  32768 |        512 |         26.8 |        0.160 |         11.65
--------------------------------------------------------------------------------
GQA 4:1 cuts the KV-cache by 75% (16 GiB -> 4 GiB at N=32768).
At 50 tok/s, MHA at N=32768 needs 859 GB/s just to read the cache: 26% of an H100's HBM bandwidth, before
any weights are read.

Columns: KV-cache = 2·L·H_kv·d·N·2 bytes · BW = cache bytes read per token × 50 tokens/s ·
H100 KV read = cache bytes ÷ 3.35 TB/s (lower bound, memory-bound decode) ·
CPU attn step = measured median time of one decoding step of ONE layer (fp32, this machine).

Results written to results/03_benchmark.json
Figure written to figures/03_kv_cache_memory.png

[OK] KV-cache transceiver online.
```

Figures



Source code

```
"""
Mission 03 — Part 3: Data & Payload Specialist
Grouped-Query Attention (GQA), a stateful KV-Cache for autoregressive decoding,
and an MHA vs GQA vs MQA memory / bandwidth / latency benchmark.

USSE Noosphere · Sector SEC-03-M45 (Pleiades) · Subsystem 03: KV-Cache Transceiver
Squadron Colby

Run:  python 03_GQA_and_KVCache_Benchmark.py            (full benchmark, ~1–2 min on a laptop CPU)
      python 03_GQA_and_KVCache_Benchmark.py --quick    (smaller latency runs)
      (writes figures/03_kv_cache_memory.png and results/03_benchmark.json)
"""
from __future__ import annotations

import argparse
import json
import math
import pathlib
import statistics
import time
from dataclasses import dataclass, asdict
from typing import Optional, Tuple

import torch
import torch.nn as nn

HERE = pathlib.Path(__file__).resolve().parent
FIG_DIR, RES_DIR = HERE / "figures", HERE / "results"

# Reference model shape: LLaMA-3-8B (32 layers, 32 query heads, head_dim 128, 8 KV heads)
N_LAYERS, N_Q_HEADS, HEAD_DIM = 32, 32, 128
BYTES_FP16 = 2
SEQ_LENGTHS = (1024, 4096, 16384, 32768)
TOKENS_PER_SEC = 50
H100_HBM_GBPS = 3350.0   # H100 SXM HBM3 bandwidth, GB/s (vendor spec)
VARIANTS = {"MHA (32:32)": 32, "GQA (32:8, 4:1)": 8, "MQA (32:1)": 1}


# ---------------------------------------------------------------------------- #
# 1. KV-Cache
# ---------------------------------------------------------------------------- #
```

```
class KVCache:
    """Pre-allocated key/value buffer for one attention layer.

    Holds (B, H_kv, max_seq_len, d) for K and V. `update` writes the new slice at
    `start_pos` and returns views of everything cached so far, so each decoding step only
    computes K, V for the NEW token instead of recomputing the whole prefix.
    """

    def __init__(self, max_batch_size: int, max_seq_len: int, n_kv_heads: int, head_dim: int,
                 dtype=torch.float32, device="cpu") -> None:
        shape = (max_batch_size, n_kv_heads, max_seq_len, head_dim)
        self.k_cache = torch.zeros(shape, dtype=dtype, device=device)
        self.v_cache = torch.zeros(shape, dtype=dtype, device=device)
        self.max_seq_len, self.seq_len = max_seq_len, 0

    def update(self, k_new: torch.Tensor, v_new: torch.Tensor,
               start_pos: int) -> Tuple[torch.Tensor, torch.Tensor]:
        b, h, n, _ = k_new.shape
        end = start_pos + n
        if end > self.max_seq_len:
            raise RuntimeError(f"KV-cache overflow: {end} > max_seq_len {self.max_seq_len}")
        self.k_cache[:b, :h, start_pos:end] = k_new
        self.v_cache[:b, :h, start_pos:end] = v_new
        self.seq_len = end
        return self.k_cache[:b, :h, :end], self.v_cache[:b, :h, :end]

    def reset(self) -> None:
        self.seq_len = 0

    @property
    def allocated_mb(self) -> float:
        return (self.k_cache.numel() + self.v_cache.numel()) * self.k_cache.element_size() / 2**20

    @property
    def used_mb(self) -> float:
        return self.allocated_mb * self.seq_len / self.max_seq_len

    def get_memory_mb(self) -> float:            # starter-compatible name
        return self.allocated_mb


# ---------------------------------------------------------------------------- #
# 2. Grouped-Query Attention
# ---------------------------------------------------------------------------- #

class GQAAttention(nn.Module):
    """Grouped-Query Attention: H_Q query heads share H_KV key/value heads (H_Q % H_KV == 0).

    H_KV == H_Q  -> Multi-Head Attention (MHA)
    H_KV == 1    -> Multi-Query Attention (MQA)
    Each group of g = H_Q / H_KV query heads attends to the same K, V head.

    `kv_expand="view"` (default) groups the queries with a reshape so K, V are broadcast
    without copying; `kv_expand="repeat"` materialises them with torch.repeat_interleave.
    Both give identical results (tested); "view" is what saves memory traffic.
    """

    def __init__(self, d_model: int = 1024, num_q_heads: int = 16, num_kv_heads: int = 4,
                 kv_expand: str = "view") -> None:
        super().__init__()
        if num_q_heads % num_kv_heads:
            raise ValueError("num_q_heads must be divisible by num_kv_heads")
        if d_model % num_q_heads:
            raise ValueError("d_model must be divisible by num_q_heads")
        self.d_model, self.h_q, self.h_kv = d_model, num_q_heads, num_kv_heads
        self.group = num_q_heads // num_kv_heads
        self.head_dim = d_model // num_q_heads
        self.kv_expand = kv_expand
        self.w_q = nn.Linear(d_model, num_q_heads * self.head_dim, bias=False)
        self.w_k = nn.Linear(d_model, num_kv_heads * self.head_dim, bias=False)
        self.w_v = nn.Linear(d_model, num_kv_heads * self.head_dim, bias=False)
        self.w_o = nn.Linear(num_q_heads * self.head_dim, d_model, bias=False)

    def attend(self, q: torch.Tensor, k: torch.Tensor, v: torch.Tensor,
               start_pos: int) -> torch.Tensor:
        """q: (B, H_q, n, d); k, v: (B, H_kv, T, d) with T = start_pos + n. Causal."""
        b, _, n, d = q.shape
        t = k.size(2)
        # causal mask for the new rows: query at absolute position start_pos+i sees keys <= it
```

```
        rows = torch.arange(start_pos, start_pos + n, device=q.device)[:, None]
        cols = torch.arange(t, device=q.device)[None, :]
        mask = torch.zeros(n, t, device=q.device).masked_fill(cols > rows, float("-inf"))
        if self.kv_expand == "repeat":
            k = torch.repeat_interleave(k, self.group, dim=1)
            v = torch.repeat_interleave(v, self.group, dim=1)
            scores = (q @ k.transpose(-2, -1)) / math.sqrt(d) + mask
            return torch.softmax(scores, dim=-1) @ v                          # (B, H_q, n, d)
        # stack the g query heads of each group as extra rows: one plain batched matmul per
        # KV head, so K and V are read once per group and never copied
        qg = q.reshape(b, self.h_kv, self.group * n, d)                       # (B, H_kv, g·n, d)
        scores = (qg @ k.transpose(-2, -1)) / math.sqrt(d) + mask.repeat(self.group, 1)
        out = torch.softmax(scores, dim=-1) @ v                               # (B, H_kv, g·n, d)
        return out.reshape(b, self.h_q, n, d)

    def forward(self, x: torch.Tensor, start_pos: int = 0,
                kv_cache: Optional[KVCache] = None) -> torch.Tensor:
        b, n, _ = x.shape
        q = self.w_q(x).view(b, n, self.h_q, self.head_dim).transpose(1, 2)
        k = self.w_k(x).view(b, n, self.h_kv, self.head_dim).transpose(1, 2)
        v = self.w_v(x).view(b, n, self.h_kv, self.head_dim).transpose(1, 2)
        if kv_cache is not None:
            k, v = kv_cache.update(k, v, start_pos)
        out = self.attend(q, k, v, start_pos if kv_cache is not None else 0)
        return self.w_o(out.transpose(1, 2).reshape(b, n, self.h_q * self.head_dim))


# ---------------------------------------------------------------------------- #
# 3. Correctness checks
# ---------------------------------------------------------------------------- #

def check_cache_matches_full_pass(seed: int = 0) -> float:
    """Token-by-token decoding with the cache must equal one full causal forward pass."""
    torch.manual_seed(seed)
    layer = GQAAttention(d_model=256, num_q_heads=8, num_kv_heads=2).eval()
    x = torch.randn(2, 12, 256)
    with torch.no_grad():
        full = layer(x)
        cache = KVCache(2, 12, 2, layer.head_dim)
        prefill = layer(x[:, :4], start_pos=0, kv_cache=cache)          # prompt of 4 tokens
        steps = [layer(x[:, t:t + 1], start_pos=t, kv_cache=cache) for t in range(4, 12)]
    inc = torch.cat([prefill, *steps], dim=1)
    return (inc - full).abs().max().item()


def check_view_equals_repeat(seed: int = 1) -> float:
    torch.manual_seed(seed)
    a = GQAAttention(256, 8, 2, kv_expand="view").eval()
    b = GQAAttention(256, 8, 2, kv_expand="repeat").eval()
    b.load_state_dict(a.state_dict())
    x = torch.randn(1, 10, 256)
    with torch.no_grad():
        return (a(x) - b(x)).abs().max().item()


def check_mha_special_case(seed: int = 2) -> float:
    """GQA with H_kv == H_q must equal plain multi-head attention from Part 1."""
    import importlib.util
    spec = importlib.util.spec_from_file_location("mha", HERE / "01_MultiHead_Attention_Core.py")
    mha_mod = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(mha_mod)
    torch.manual_seed(seed)
    mha = mha_mod.MultiHeadSelfAttention(d_model=128, num_heads=4).eval()
    gqa = GQAAttention(128, 4, 4).eval()
    gqa.load_state_dict({"w_q.weight": mha.w_q.weight, "w_k.weight": mha.w_k.weight,
                         "w_v.weight": mha.w_v.weight, "w_o.weight": mha.w_o.weight})
    x = torch.randn(2, 9, 128)
    with torch.no_grad():
        return (mha(x, causal=True)[0] - gqa(x)).abs().max().item()


# ---------------------------------------------------------------------------- #
# 4. Benchmark
# ---------------------------------------------------------------------------- #

def kv_bytes(n_tokens: int, n_kv_heads: int, n_layers: int = N_LAYERS, head_dim: int = HEAD_DIM,
             batch: int = 1, bytes_per: int = BYTES_FP16) -> int:
    """K and V for every layer: 2 · L · H_kv · d_head · N · B · bytes."""
    return 2 * n_layers * n_kv_heads * head_dim * n_tokens * batch * bytes_per
```

```
@dataclass
class Row:
    variant: str
    seq_len: int
    kv_heads: int
    kv_cache_mb: float
    bandwidth_gb_s_at_50tps: float
    h100_kv_read_ms_per_token: float
    cpu_attn_ms_per_token_per_layer: Optional[float]


def measure_decode_ms(seq_len: int, n_kv_heads: int, repeats: int = 9) -> float:
    """Median wall time of ONE decoding step of one attention layer with a full cache.

    fp32 on CPU, d_model = 32 · 128 = 4096 (LLaMA-3-8B width). Projections included.
    """
    torch.manual_seed(0)
    layer = GQAAttention(N_Q_HEADS * HEAD_DIM, N_Q_HEADS, n_kv_heads).eval()
    cache = KVCache(1, seq_len + 1, n_kv_heads, HEAD_DIM)
    cache.k_cache.normal_(); cache.v_cache.normal_()
    x = torch.randn(1, 1, N_Q_HEADS * HEAD_DIM)
    times = []
    with torch.no_grad():
        for _ in range(2):                                          # warm-up
            layer(x, start_pos=seq_len, kv_cache=cache)
        for _ in range(repeats):
            t0 = time.perf_counter()
            layer(x, start_pos=seq_len, kv_cache=cache)
            times.append((time.perf_counter() - t0) * 1000)
    del cache
    return statistics.median(times)


def run_benchmark(measure: bool = True, max_measure_len: int = 32768) -> list[Row]:
    rows: list[Row] = []
    for name, h_kv in VARIANTS.items():
        for n in SEQ_LENGTHS:
            b = kv_bytes(n, h_kv)
            ms = measure_decode_ms(n, h_kv) if measure and n <= max_measure_len else None
            rows.append(Row(name, n, h_kv, b / 2**20, b * TOKENS_PER_SEC / 1e9,
                            b / (H100_HBM_GBPS * 1e9) * 1000, ms))
    return rows


def print_table(rows: list[Row]) -> None:
    print("KV-cache for a LLaMA-3-8B-shaped model (32 layers, 32 query heads, d_head 128, fp16, batch 1)")
    hdr = (f"{'Variant':<17}| {'N':>6} | {'KV-cache':>10} | {'BW @50 tok/s':>12} | "
           f"{'H100 KV read':>12} | {'CPU attn step':>13}")
    print(hdr); print(f"{'':<17}| {'':>6} | {'(MB)':>10} | {'(GB/s)':>12} | {'(ms/token)':>12} | "
                      f"{'(ms/layer)':>13}")
    print("-" * len(hdr))
    for r in rows:
        cpu = f"{r.cpu_attn_ms_per_token_per_layer:>13.2f}" if r.cpu_attn_ms_per_token_per_layer \
            is not None else f"{'—':>13}"
        print(f"{r.variant:<17}| {r.seq_len:>6} | {r.kv_cache_mb:>10,.0f} | "
              f"{r.bandwidth_gb_s_at_50tps:>12.1f} | {r.h100_kv_read_ms_per_token:>12.3f} | {cpu}")
    print("-" * len(hdr))
    mha = {r.seq_len: r for r in rows if r.kv_heads == 32}
    gqa = {r.seq_len: r for r in rows if r.kv_heads == 8}
    n = SEQ_LENGTHS[-1]
    print(f"GQA 4:1 cuts the KV-cache by {100 * (1 - gqa[n].kv_cache_mb / mha[n].kv_cache_mb):.0f}% "
          f"({mha[n].kv_cache_mb / 1024:.0f} GiB -> {gqa[n].kv_cache_mb / 1024:.0f} GiB at N={n}).")
    over = [r for r in rows if r.bandwidth_gb_s_at_50tps > H100_HBM_GBPS]
    if not over:
        print(f"At 50 tok/s, MHA at N={n} needs {mha[n].bandwidth_gb_s_at_50tps:.0f} GB/s just to read the "
              f"cache: {100 * mha[n].bandwidth_gb_s_at_50tps / H100_HBM_GBPS:.0f}% of an H100's HBM "
              f"bandwidth, before any weights are read.")


def cache_vs_recompute(n: int = 256, seed: int = 0) -> Tuple[float, float]:
    """Wall time to generate n tokens with the cache vs recomputing the full prefix each step."""
    torch.manual_seed(seed)
    layer = GQAAttention(512, 8, 2).eval()
    x = torch.randn(1, n, 512)
    with torch.no_grad():
        t0 = time.perf_counter()
        for t in range(1, n + 1):
            layer(x[:, :t])                                 # no cache: whole prefix again
        t_no = time.perf_counter() - t0
```

```
        cache = KVCache(1, n, 2, layer.head_dim)
        t0 = time.perf_counter()
        for t in range(n):
            layer(x[:, t:t + 1], start_pos=t, kv_cache=cache)  # one new token per step
        t_kv = time.perf_counter() - t0
    return t_no * 1000, t_kv * 1000


def plot_memory(rows: list[Row]) -> None:
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError:
        print("(matplotlib not installed: skipping figure)")
        return
    FIG_DIR.mkdir(exist_ok=True)
    fig, ax = plt.subplots(figsize=(6.6, 4.2))
    for name, h in VARIANTS.items():
        rs = [r for r in rows if r.kv_heads == h]
        ax.plot([r.seq_len for r in rs], [r.kv_cache_mb / 1024 for r in rs], "o-", label=name)
    ax.axhline(80, color="grey", ls=":", lw=1); ax.text(1100, 82, "H100 80 GB", color="grey", fontsize=8)
    ax.set_xscale("log", base=2); ax.set_yscale("log", base=2)
    ax.set_xlabel("context length N (tokens)"); ax.set_ylabel("KV-cache per sequence (GiB)")
    ax.set_title("KV-cache memory, LLaMA-3-8B shape, fp16")
    ax.legend(); fig.tight_layout(); fig.savefig(FIG_DIR / "03_kv_cache_memory.png", dpi=150)
    plt.close(fig)
    print("Figure written to figures/03_kv_cache_memory.png")


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--quick", action="store_true", help="measure CPU latency only up to N=4096")
    ap.add_argument("--no-measure", action="store_true", help="skip CPU latency measurement")
    args = ap.parse_args()

    print("USSE NOOSPHERE · SUBSYSTEM 03 · PART 3: GQA & KV-CACHE\n")
    print("=== Correctness ===")
    e1, e2, e3 = check_cache_matches_full_pass(), check_view_equals_repeat(), check_mha_special_case()
    print(f"cached token-by-token decoding vs full causal pass: max |diff| = {e1:.2e}")
    print(f"GQA grouped view vs repeat_interleave:              max |diff| = {e2:.2e}")
    print(f"GQA with H_kv = H_q vs Part 1 MultiHeadSelfAttention: max |diff| = {e3:.2e}")
    assert max(e1, e2, e3) < 1e-4
    print("[PASS] KV-cache and GQA are exact.\n")

    t_no, t_kv = cache_vs_recompute()
    print("=== Why cache? Generating 256 tokens with one layer (CPU) ===")
    print(f"recompute prefix every step: {t_no:,.0f} ms   ·   with KV-cache: {t_kv:,.0f} ms   "
          f"·   {t_no / t_kv:.1f}× faster\n")

    print("=== Benchmark: MHA vs GQA vs MQA ===")
    rows = run_benchmark(measure=not args.no_measure,
                         max_measure_len=4096 if args.quick else SEQ_LENGTHS[-1])
    print_table(rows)
    print("\nColumns: KV-cache = 2·L·H_kv·d·N·2 bytes · BW = cache bytes read per token × 50 tokens/s ·\n"
          "H100 KV read = cache bytes ÷ 3.35 TB/s (lower bound, memory-bound decode) ·\n"
          "CPU attn step = measured median time of one decoding step of ONE layer (fp32, this machine).\n")
    RES_DIR.mkdir(exist_ok=True)
    (RES_DIR / "03_benchmark.json").write_text(json.dumps(
        {"model": {"layers": N_LAYERS, "q_heads": N_Q_HEADS, "head_dim": HEAD_DIM, "dtype": "fp16"},
         "tokens_per_sec": TOKENS_PER_SEC, "h100_hbm_gbps": H100_HBM_GBPS,
         "cache_vs_recompute_ms": {"recompute": t_no, "kv_cache": t_kv},
         "rows": [asdict(r) for r in rows]}, indent=2))
    print("Results written to results/03_benchmark.json")
    plot_memory(rows)
    print("\n[OK] KV-cache transceiver online.")


if __name__ == "__main__":
    main()
```